

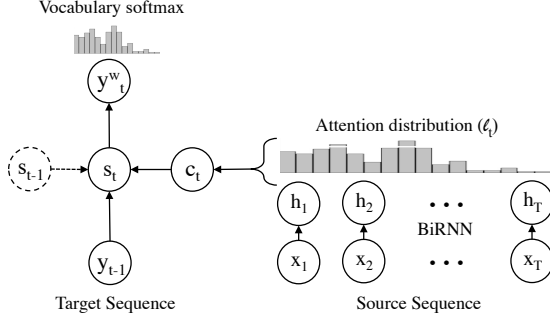


Figure 2: A depiction of neural machine translation architecture with attention. At each timestep, the model generates the attention distribution $\mathbf{l}_t$. We use $\mathbf{l}_t$ and the encoder’s hidden states to obtain the context $\mathbf{c}_t$. The decoder uses $\mathbf{c}_t$ to predict a vector of probabilities for the words $\mathbf{w}_t$ by using vocabulary softmax.

4 The Pointer Softmax

In this section, we introduce our method, called as the pointer softmax (PS), to deal with the rare and unknown words. The pointer softmax can be an applicable approach to many NLP tasks, because it resolves the limitations about unknown words for neural networks. It can be used in parallel with other existing techniques such as the large vocabulary trick (Jean et al., 2014). Our model learns two key abilities jointly to make the pointing mechanism applicable in more general settings: (i) to predict whether it is required to use the pointing or not at each time step and (ii) to point any location of the context sequence whose length can vary widely over examples. Note that the pointer networks (Vinyals et al., 2015) are in lack of the ability (i), and the ability (ii) is not achieved in the models by (Luong et al., 2015).

To achieve this, our model uses two softmax output layers, the *shortlist* softmax and the *location* softmax. The shortlist softmax is the same as the typical softmax output layer where each dimension corresponds a word in the predefined word shortlist. The location softmax is a pointer network where each of the output dimension corresponds to the location of a word in the context sequence. Thus, the output dimension of the location softmax varies according to the length of the given context sequence.

At each time-step, if the model decides to use the shortlist softmax, we generate a word w_t from the shortlist. Otherwise, if it is expected that the context sequence contains a word which needs to

be generated at the time step, we obtain the location of the context word l_t from the location softmax. The key to making this possible is deciding when to use the shortlist softmax or the location softmax at each time step. In order to accomplish this, we introduce a switching network to the model. The switching network, which is a multilayer perceptron in our experiments, takes the representation of the context sequence (similar to the input annotation in NMT) and the previous hidden state of the output RNN as its input. It outputs a binary variable z_t which indicates whether to use the shortlist softmax (when $z_t = 1$) or the location softmax (when $z_t = 0$). Note that if the word that is expected to be generated at each time-step is neither in the shortlist nor in the context sequence, the switching network selects the shortlist softmax, and then the shortlist softmax predicts *UNK*. The details of the pointer softmax model can be seen in Figure 3 as well.

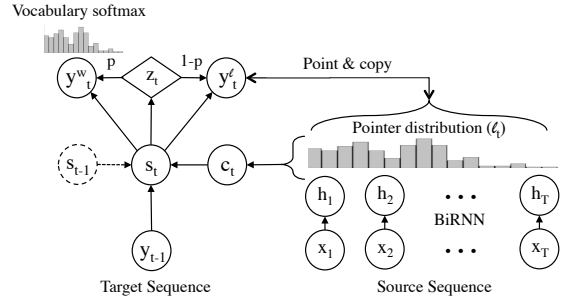


Figure 3: A depiction of the Pointer Softmax (PS) architecture. At each timestep, $\mathbf{l}_t$, $\mathbf{c}_t$ and $\mathbf{w}_t$ for the words over the limited vocabulary (shortlist) is generated. We have an additional switching variable z_t that decides whether to use vocabulary word or to copy a word from the source sequence.

More specifically, our goal is to maximize the probability of observing the target word sequence $\mathbf{y} = (y_1, y_2, \dots, y_{T_y})$ and the word generation source $\mathbf{z} = (z_1, z_2, \dots, z_{T_y})$, given the context sequence $\mathbf{x} = (x_1, x_2, \dots, x_{T_x})$:

$$p_{\theta}(\mathbf{y}, \mathbf{z}|\mathbf{x}) = \prod_{t=1}^{T_y} p_{\theta}(y_t, z_t | y_{<t}, z_{<t}, \mathbf{x}). \quad (4)$$

Note that the word observation y_t can be either a word w_t from the shortlist softmax or a location l_t from the location softmax, depending on the switching variable z_t .

Considering this, we can factorize the above